# Python Engineering - Nota de referencia

## __name__, __main__ y el main guard

Explicacion simple de por que suele aparecer `if __name__ == "__main__":` al final de muchos programas Python.

## 1. Que es __name__?

Python crea automaticamente una variable especial llamada `__name__` en cada modulo. Su valor depende de como se haya cargado el archivo.

| Situacion | Valor de __name__ |
|---|---|
| Ejecutas python main.py | '__main__' |
| Otro archivo hace import vertical_motion | 'vertical_motion' |

## 2. Que pregunta el main guard?

```
if __name__ == '__main__':
    main()
```

Pregunta: **"Este archivo es el programa que se esta ejecutando directamente?"** Si si, llama a `main()`. Si solo se ha importado, no arranca automaticamente el programa.

## 3. Por que se usa normalmente asi?

Porque un mismo archivo puede funcionar como programa ejecutable y como modulo reutilizable. El main guard evita efectos secundarios al importar: las funciones quedan disponibles, pero el flujo principal no se ejecuta por sorpresa.

## 4. Por que ademas usar def main()?

Encapsula el flujo principal, organiza el codigo, facilita pruebas y deja claro el punto de entrada de la aplicacion.

```
def main():
    config = {...}
    results = simulate_vertical_motion(config)
    print(results)

if __name__ == '__main__':
    main()
```

## 5. Modelo mental

```
DIRECT EXECUTION
python main.py -> __name__ = '__main__' -> main() RUNS

IMPORT
import main -> __name__ = 'main' -> main() DOES NOT RUN AUTOMATICALLY
```

## 6. Importante

`__main__` no es el nombre obligatorio de una funcion y `main()` no es una palabra reservada. `__main__` es un valor especial de `__name__`; llamar `main()` a la funcion principal es una convencion muy extendida.

Regla practica: usa el main guard cuando quieras ejecutar un flujo principal directamente y, al mismo tiempo, poder importar las funciones del archivo sin lanzar ese flujo.